

WEEK 3

SELLER AND INVENTORY MANAGEMENT SYSTEM

HEMAVATHI.B (E25AI114)

Week 3 Task Report

Scenario

The Perfume shop sells products supplied by multiple sellers. The system should maintain seller details, track the products supplied by each seller, and monitor inventory levels to ensure product availability.

1. Create Seller and Inventory Tables

```
SQL> CREATE TABLE Seller(Seller_ID VARCHAR(7) , Seller_Name VARCHAR(25), Email VARCHAR(50), Phone NUMBER(10), Address VARCHAR (100));
Table created.

SQL> CREATE TABLE Inventory(Inv_ID VARCHAR(10) PRIMARY KEY, Prod_ID VARCHAR(10), Sell_ID VARCHAR(7), Stock NUMBER,
2 Stock_Status VARCHAR(20), Last_Upd DATE);
Table created.
```

2. Establish Relationships

Seller and Product

One seller can supply multiple products. One product can be supplied by one or multiple sellers.

Relationship: Seller -> Product is one to many.

Product and Inventory

Each product has inventory information. Inventory is used to track the available stock.

Relationship: Product -> Inventory is one to one.

Primary Key added for Both Tables

```
SQL> ALTER TABLE Seller ADD CONSTRAINT PK_Seller PRIMARY KEY(Seller_ID);
Table altered.

SQL> ALTER TABLE Inventory ADD CONSTRAINT PK_Inv PRIMARY KEY(Inventory_ID);
Table altered.
```

Add an appropriate primary key to both tables so that each record is uniquely identified.

Foreign Key for Inventory Table

The inventory table should contain a foreign-key reference to the related product, linking product information with its stock details.

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Prod FOREIGN KEY(Product_ID) REFERENCES Product(Product_ID);  
Table altered.  
  
SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Seller FOREIGN KEY(Seller_ID) REFERENCES Seller(Seller_ID);  
Table altered.
```

Constraints to make Unique Email ID

```
SQL> ALTER TABLE Seller ADD CONSTRAINT Un_mail UNIQUE(Email);  
  
Table altered.
```

Apply a UNIQUE constraint to the seller email column so duplicate email IDs are not accepted.

To make Email NOT NULL

```
SQL> ALTER TABLE Seller MODIFY Email NOT NULL;  
  
Table altered.
```

Add Constraint to Check the Stock Quantity is Positive

Use a CHECK constraint to ensure that stock quantity is not negative and only valid inventory values are entered.

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT CK_Quan CHECK(Stock_Quantity >=0);  
  
Table altered.
```

Insert Values to Seller Table

Insert sample seller records into the Seller table for testing and report generation.

```

SQL> INSERT INTO Seller VALUES ('&Seller_ID','&Seller_Name','&Email','&Phone','&Address');
Enter value for seller_id: S001
Enter value for seller_name: Anand Kumar
Enter value for email: anand@gmail.com
Enter value for phone: 9876543210
Enter value for address: Chennai
old 1: INSERT INTO Seller VALUES ('&Seller_ID','&Seller_Name','&Email','&Phone','&Address')
new 1: INSERT INTO Seller VALUES ('S001','Anand Kumar','anand@gmail.com','9876543210','Chennai')

1 row created.

```

```
SQL> SELECT * FROM Seller;
```

```
SELLER_ SELLER_NAME
```

```
EMAIL
```

```
PHONE
```

```
ADDRESS
```

```

S001    Anand Enterprises
anand@gmail.com
Chennai

```

```
9876543210
```

```

S002    Priya Traders
priya@gmail.com
Tambaram

```

```
9876543211
```

```
SELLER_ SELLER_NAME
```

```
EMAIL
```

```
PHONE
```

```
ADDRESS
```

```

S003    Vijaya Enterprises
vijaya@gmail.com
Chromepet

```

```
9876543212
```

```

S004    Sri Krishna & Co
srikrishna@gmail.com

```

```
9876543213
```

Insert Values to Inventory Table

Insert sample inventory values associated with the products.

```
SQL> INSERT INTO Inventory VALUES ('I001','BR-01','S003',50,'AVAILABLE',SYSDATE);
```

```
1 row created.
```

```
SQL>
```

```
SQL> INSERT INTO Inventory VALUES ('I002','BR-02','S004',35,'AVAILABLE',SYSDATE);
```

```
1 row created.
```

INV_ID	BRAND_ID	SELL_ID	STOCK	STOCK_STATUS	LAST_UPD
I001	BR-01	S003	50	AVAILABLE	22-SEP-26
I002	BR-02	S004	35	AVAILABLE	22-SEP-26
I004	BR-04	S006	60	AVAILABLE	22-SEP-26
I005	BR-05	S007	10	AVAILABLE	22-SEP-26
I006	BR-06	S003	45	AVAILABLE	22-SEP-26
I007	BR-07	S004	25	AVAILABLE	22-SEP-26
I008	BR-08	S005	40	AVAILABLE	22-SEP-26

```
7 rows selected.
```

Report 1: Seller-wise Product Report

Display:

- Seller name
- Number of products supplied
- Available stock

Use JOIN operations to get the required

```
SQL> SELECT s.SELLER_NAME, COUNT(i.INV_ID) AS NUMBER_OF_PRODUCTS, SUM(i.STOCK) AS AVAILABLE_STOCK FROM Seller s JOIN Inventory i ON s.SELLER_ID = i.SELL_ID WHERE i.STOCK_STATUS = 'AVAILABLE' GROUP BY s.SELLER_NAME;
```

SELLER_NAME	NUMBER_OF_PRODUCTS	AVAILABLE_STOCK
Sakthi Enterprises	1	60
Vijaya Enterprises	2	95
Sri Krishna & Co	2	60
Mahalakshmi Traders	1	40
Sri Devi & Co	1	10

Report 2: Stock Availability Report

Display:

- Product name
- Stock quantity
- Stock status

```
SQL> SELECT b.BRAND_NAME AS PRODUCT_NAME, i.STOCK AS STOCK_QUANTITY, i.STOCK_STATUS FROM Brand b JOIN Inventory i ON b.BRAND_ID = i.BRAND_ID WHERE i.STOCK_STATUS = 'AVAILABLE';
```

PRODUCT_NAME	STOCK_QUANTITY	STOCK_STATUS
Dior	50	AVAILABLE
Chanel	35	AVAILABLE
Versace	60	AVAILABLE
Armani	10	AVAILABLE
Burberry	45	AVAILABLE
Calvin	25	AVAILABLE
Hugo Boss	40	AVAILABLE

7 rows selected.

Report 3: Inventory Analysis

Find:

- Total products available
- Products out of stock
- Highest stocked products
- Average inventory quantity

```
SQL> SELECT COUNT(*) AS TOTAL_PRODUCTS_AVAILABLE FROM Inventory WHERE STOCK_STATUS = 'AVAILABLE';
```

TOTAL_PRODUCTS_AVAILABLE
7

```
SQL> SELECT COUNT(*) AS PRODUCTS_OUT_OF_STOCK FROM Inventory WHERE STOCK_STATUS = 'OUT OF STOCK';
```

PRODUCTS_OUT_OF_STOCK
0

```
SQL> SELECT b.BRAND_NAME AS PRODUCT_NAME, i.STOCK AS HIGHEST_STOCK FROM Brand b JOIN Inventory i ON b.BRAND_ID = i.BRAND_ID WHERE i.STOCK = (SELECT MAX(STOCK) FROM Inventory);
```

PRODUCT_NAME	HIGHEST_STOCK
Versace	60

```
SQL> SELECT AVG(STOCK) AS AVERAGE_INVENTORY_QUANTITY FROM Inventory;
```

AVERAGE_INVENTORY_QUANTITY
37.8571429

END OF WEEK 3 REPORT

